Лабораторная работа №3 Наследование

Цель работы: Изучить принципы наследования. Простое и множественное наследование.

Краткие теоретические сведения

Наследование – это механизм получения нового класса из существующего. Существующий класс может быть дополнен или изменен для создания производного класса. При создании нового класса вместо написания полностью новых данных и функций программист может указать, что новый класс должен наследовать данные и функции ранее определенного базового класса. Этот новый класс называется производным классом. Каждый производный класс сам является кандидатом на роль базового класса для будущих производных классов. При простом наследовании класс порождается одним базовым классом. При множественном наследовании производный класс наследуется несколькими базовыми классами. Производный класс обычно добавляет свои данные и функции, так что производный класс в общем случае больше своего базового.

Шаблон объявления производного класса можно представить следующим образом:

ключ_класса имя_производного_класса:необязательный_модификатор_доступа имя_базового_класса
{
    Тело производного класса
};

Пример:

```
class Location
{
   int x,y;
     public:
        . . .
};
class Point:public Location
{
   Тело класса Point
};
```

Класс Location является базовым и наследуется с атрибутом public. Класс Point – производный класс. Двоеточие (:) отделяет производный класс от базового. Атрибут класса (модификатор прав доступа) может задаваться ключевыми словами public и private. Атрибут может опускаться – в этом случае принимается атрибут по умолчанию (для ключевого слова class – private, для struct – public). Объединение (union) не может быть базовым или производным классом.

Модификатор прав доступа используется для изменения прав доступа к наследуемым элементам класса в соответствии с правилами:

Ограничения на доступ в базовом классе	Модификатор наследования прав	Ограничения на доступ в производном классе
private	private	Нет доступа
protected	private	private
public	private	private
private	public	Нет доступа
protected	public	protected
public	public	public

Отметим, что в производных классах права на доступ к элементам базовых классов не могут быть расширены, а только более ограничены.

Пример:

```
class A
{
        int a1;
    public:
        int a2;
        void f1(void);
};
class B:A
{
        int b1;
    public:
   void f1(void)
   {
        a1=1;  // ошибка, a1–private-переменная класса A и доступна только для
                    // методов и дружественных функций собственного класса
        b1=0;       // доступ к переменной типа private из метода класса
        a2=1;       // a2 унаследована из класса A с атрибутом доступа private и
                   // поэтому доступна в методе класса
   }
};
void main(void)
{
   A a_ob1;            // объявление объекта a_ob1 класса A
   B b_ob1;            // объявление объекта  b_ob1 класса B
   b_ob1.a2+=1;     // ошибка, т.к. a2 private
   a_ob1.a2+=1;     // допустимая операция
}
```

Рассмотрим еще пример, демонстрирующий наследование прав доступа к элементам базовых классов.

```
   #include<iostream>
#include <iomanip>
```

```
using namespace std;
#include<string.h>
#define N 10

class book
{
   protected:
       char naz[20];               // название книги
       int kl;                     // количество страниц
   public:
       book(char *, int);          // конструктор класса book
       ~book();                    // деструктор класса book
};
class prog_book: public book
{
       char fm[10];                // фамилия автора
   public:
       prog_book(char *, int, char *);  // конструктор класса prog_b
       ~prog_book();                     // деструктор класса avt
       void see();
};
class OS_book: public book
{
       char naz_OS[10];                // назапние ОС
   public:
       OS_book(char *, int, char *);  // конструктор класса prog_b
       ~OS_book();                    // деструктор класса avt
       void see();
};
book::book(char *s1, int i)
{
       cout << "Constr book  " << endl;
       strcpy_s(naz,strlen(s1)+1,s1);
       kl=i;
}
book::~book()
{
       cout << "Destr book  " << endl;
}
prog_book::prog_book(char *s1, int i, char *s2):book(s1,i)
{
       cout << "Constr prog_book  " << endl;
       strcpy_s(fm,strlen(s2)+1,s2);
}
 prog_book::~prog_book()
{      cout << "Destr prog_book  " << endl;
}
void prog_book::see()
{
       cout << setw(15) << "Nazv book" << setw(15) << "kol_vo str" << setw(10) << "Avtor"
<< endl;
       cout << setw(15) << naz << setw(15) << kl << setw(10) << fm << endl;
}

OS_book::OS_book(char *s1, int i, char *s2):book(s1,i)
{
       cout << "Constr OS_book  " << endl;
       strcpy_s(naz_OS,strlen(s2)+1,s2);
}
 OS_book::~OS_book()
{      cout << "Destr OS_book  " << endl;
}
void OS_book::see()
{
```

```
	cout << setw(15) << "Nazv book" << setw(15) << "kol_vo str" << setw(10)  <<
"naz_OS"  << endl;
	cout <<setw(15) << naz << setw(15) <<  kl << setw(10) << naz_OS << endl;
}
void main(void)
{
	prog_book obj("C++",1500, "Deitel");
	OS_book obj1("OS", 1000, "Android");
	obj.see();
	obj1.see();

}
```

Если базовый класс имеет конструктор с одним или более аргументами, то любой производный класс должен иметь конструктор. В нашем примере конструктор класса book задан в виде:

```
book::book(char *s1, int i)
{
	cout << "Constr book  " << endl;
	strcpy_s(naz,strlen(s1)+1,s1);
	kl=i;
}
```

Множественное наследование – это когда производный класс образуется из нескольких базовых классов.

В языке C++ допускается образовывать производный класс от нескольких базовых классов. Поэтому общий синтаксис описания конструктора производного класса представляется в виде:

имя_конструктора_производного_класса(список аргументов для производного класса): имя_базового_класса1(список аргументов для конструктора базового класса1), … , имя_базового_классаN(список аргументов для базового классаN)

{

	тело конструктора

}

Пример:

```
#include<iostream>
using namespace std;
class Base_1		// базовый класс Base_1
{
	int a;
	protected:
	int b;
	 public:
	Base_1(int x, int y);
	~Base_1();


	void show1(void)
	{
		cout << "a=" << a << "b=" << b;
	}
};
class Base_2		// базовый класс Base_2
{
	 protected:
	int c;
	 public:
```

```
	Base_2(int r);
	~Base_2();
	void show2(void)
	{
		cout << "c=" << c << endl;
	}
};
class Derive: public Base_1, public Base_2	// производный класс Derive
{
	int p;
	 public:
	Derive(int x, int y, int z);
	~Derive();
	void show3(void)
	{
		cout << "a+b+c=" << p+b+c << endl;
	}
};
Base_1::Base_1(int x, int y)
{
	a=x;
	b=y;
	cout << "Constr Base_1" << endl;
}
Base_1::~Base_1()
{
	cout << "Destr Base_1" << endl;
}
 Base_2::Base_2(int x)
{
	c=x;
	cout << " Constr Base_2" << endl;
}
Base_2::~Base_2()
{
	cout << "Destr Base_2" << endl;
}
Derive::Derive(int x, int y, int z):Base_1(x,y), Base_2(z)
{
	p=x;
	cout << "Constr  Derive" << endl;
}
Derive::~Derive()
{
	cout << "Destr  Derive" << endl;
}
void main(void)
{
	int x,y,z;
	cout << "Vvod x, y, z" << endl;
	cin >> x >> y >> z;	// ввод значений переменных
	Derive my_d(x,y,z);	// объявление объекта
	my_d.show1();
	my_d.show2();
	my_d.show3();
}
```

Каждый из классов Base_1 и Base_2 содержит свой конструктор с параметрами. В связи с этим производный класс тоже должен содержать свой конструктор с параметрами. В функции main() объявлен объект my_d. В результате будут вызваны конструкторы базовых и производного классов. Конструктор производного класса

Derive::Derive(int x, int y, int z):Base_1(x,y), Base_2(z)
{ … }

Последовательно вызываются конструкторы базовых классов в соответствии с их появлением в списке, т.е. сначала вызывается конструктор Base_1, затем конструктор Base_2, после вызывается конструктор производного класса Derive. Выполнение деструкторов осуществляется в обратном порядке.

Виртуальные функции

Один из основных принципов объектно-ориентированного программирования предполагает использование идеи «один интерфейс – множество методов реализации». Эта идея заключается также в том, что базовый класс обеспечивает все элементы, которые производные классы могут непосредственно использовать, плюс набор функций, которые производные классы должны реализовать путем их переопределения. Наряду с механизмом перегрузки функций это достигается использованием виртуальных (virtual) функций. *Виртуальная* функция – это функция, объявленная с ключевым словом virtual в базовом классе и переопределенная в одном или нескольких производных от этого классах. При вызове объекта базового или производных классов динамически (во время выполнения программы) определяется, какую из функций требуется вызвать, основываясь на типе объекта.

Рассмотрим пример использования виртуальной функции.

```
#include<iostream>
#include<iomanip>
#include<string.h>
using namespace std;
class base                    //  базовый класс
{
protected:
       int h;
 public:
        base(int h)
        {
               cout << "Constr base" << endl;
               this->h=h;
        }
        ~base()
       {
              cout << "Destr base" << endl;
       }
       virtual const char * name();
       virtual double area();
};
 const char * base::name()
{

              return "noname";
}
double base::area()
{
       return 0;
}
class rect: public base            // производный класс “прямоугольник”
{
```

```
	int h1;                     // размеры прямоугольника
	//int s;
public:
	rect(int H, int S):base(H)  // конструктор
	{
		cout << "Constr rect" << endl;
		h1=S;
	}
	~rect()
	{
		cout << "Destr rect" << endl;
	}
	virtual const char * name() override; // вывод на экран названия фигуры
	virtual double area()  override;      // вывод на экран площади фигуры
};
const char * rect::name()                 // вывод на экран названия фигуры
{
	return "Rect";
}
double rect::area()                       // вывод на экран площади фигуры
{
	return h*h1;
}
class circl: public base                  // производный класс “окружность”
{
	//int r;                              // радиус окружности
public:
	circl(int R):base(R)                  // конструктор
	{
		cout << "Constr circl" << endl;
	}
	~circl()
	{
		cout << "Destr circl" << endl;
	}
virtual const char * name();     // вывод на экран названия фигуры
virtual double area();           // вывод на экран площади фигуры
};
const char * circl::name()       // вывод на экран названия фигуры
{
	return "Circle";
}
double circl::area()             // вывод на экран площади фигуры
{
	return 3.14*h*h;
}
int main()
{
	int a,b;
	base *p[2];                  //  массив указателей на базовый класс
	cout << "Vvod a,d" << endl;
	cin >> a >> b;
	/*rect obj1(a,b);
	obj1.name();
	obj1.area();
	circl obj2(a);
	p[0]=&obj1;
	p[1]=&obj2;*/
	p[0]=new rect(a,b);
	p[1]=new circl(a);
	for(int i=0; i < 2; i++)
		cout  << "area  " << p[i]->name() << setw(10) << p[i]->area() << endl;
}
```

Объявление virtual void name(void) говорит о том, что функция name() может быть различной для базового и производных классов. Тип виртуальной функции не может быть переопределен в производных классах. Исключением является случай, когда возвращаемый тип виртуальной функции является указателем или ссылкой на порожденный класс, а виртуальная функция основного класса - указателем или ссылкой на базовый класс. В производных классах функция может иметь список параметров, отличный от параметров виртуальной функции базового класса. В этом случае эта функция будет не виртуальной, а перегруженной. Вызов функций должен производиться с учетом списка параметров.

Механизм вызова виртуальных функций можно пояснить следующим образом. При создании нового объекта для него выделяется память. Для виртуальных функций (и только для них) создается указатель на таблицу функций, из которой выбирается требуемая в процессе выполнения. Доступ к виртуальной функции осуществляется через этот указатель и соответствующую таблицу (то есть выполняется косвенный вызов функции).

Если функция вызывается с использованием ее полного имени obj1.name(); то виртуальный механизм игнорируется.

Если функция была объявлена как виртуальная в некотором классе (базовом классе), то она остается виртуальной независимо от количества уровней в иерархии классов, через которые она прошла.

```
class A
{  . . .
   public:  virtual void fun() {}
};
class B : public A
{  . . .
   public:  void fun() {}
};

class C : public B
{  . . .
   public:
   . . .   // в объявлении класса C отсутствует описание функции fun()
};
main()
{ A *p=&a;
  B b;
  C c;
  p->fun();    // вызов версии виртуальной функции fun для класса A
  p=&b;
  p->fun();    // вызов версии виртуальной функции fun для класса B
  p=&c;
  p->fun();    // вызов версии виртуальной функции fun для класса C
}
```

Если в производном классе виртуальная функция не переопределяется, то используется ее версия из базового класса.

```
class A
{  . . .
   public:  virtual void fun() {}
};
class B : public A
{  . . .
   public:  void fun() {}
};
class C : public B
{  . . .
   public:  void fun() {}
};
main()
{ A *p=&a;
  B b;
  C c;
  p->fun();    // вызов версии виртуальной функции fun для класса A
  p=&b;
  p->fun();    // вызов версии виртуальной функции fun для класса B
  p=&c;
  p->fun();    // вызов версии виртуальной функции fun для класса A
}
```

Основное достоинство данной иерархии состоит в том, что код не нуждается в глобальном изменении, даже при добавлении вычислении площадей новых фигур. Изменения вносятся локально и благодаря полиморфному характеру кода распространяются автоматически.

Перечислим основные свойства и правила использования виртуальных функций:

-виртуальный механизм поддерживает полиморфизм на этапе выполнения программы. Это значит, что требуемая версия программы выбирается на этапе выполнения программы, а не компиляции;

- класс, содержащий хотя бы одну виртуальную функцию, называется полиморфным;

- виртуальные функции можно объявить только в классах (class) и структурах (struct);

- виртуальными функциями могут быть только нестатические функции (без спецификатора static), так как характеристика virtual унаследуется. Функция порожденного класса автоматически становится virtual;

- виртуальные функции можно объявить со спецификатором friend для другого класса;

- виртуальными функциями могут быть только не глобальные функции (то есть компоненты класса);

- если виртуальная функция объявлена в производном классе со спецификатором virtual, то можно рассматривать новые версии этой функции в классах, наследуемых из этого производного класса. Если спецификатор virtual опущен, то новые версии функции далее не будут рассматриваться;
- для вызова виртуальной функции требуется больше времени, чем для не виртуальной. При этом также требуется дополнительная память для хранения виртуальной таблицы;
- при использовании полного имени при вызове некоторой виртуальной функции (например, grup ::see();), виртуальный механизм не применяется.

Порядок выполнения работы

1. Изучить краткие теоретические сведения.
2. Ознакомиться с материалами лекций 10,11.
3. Разработать алгоритм программы и диаграмму классов.
4. Написать, отладить и выполнить программу.

Варианты заданий

1. Реализовать класс «Статья» с полями «название статьи» и «имя автора» и методами, позволяющими посмотреть эти поля. Реализовать класс «Библиотечная карточка» хранящая автора, название, авторский знак, инвентарный номер, код по тематическому каталогу. Класс должен иметь методы, возвращающие эти поля и иметь метод для алфавитного поиска, возвращающий объединенное значение полей «автор» и «название».

Реализовать производные от него классы:

1) «Карточка для самостоятельного издания» с методами, возвращающими издательство, год издания, тираж, количество страниц и производные от него классы:

а) «карточка для сборника статей» с методом, возвращающим карточки, описывающие статьи (см. ниже);

б) «карточка для книги» (книга состоит из одной статьи), дополнительный метод должен возвращать статью.

2) «Карточка для статьи» с методами, возвращающими кроме имени автора, статью и ссылку на карточку для издания, в котором статья опубликована. Реализовать класс «Каталог», с производными «Тематический каталог», «Алфавитный каталог» с методами, реализующими поиск по шифру или автору (названию).

2. При наличии классов Circle(круг), Square (квадрат) и Triangle(треугольник), производных от Shape(фигура), определите функцию intersect() (пересечение), которая принимает две фигуры Shape* в качестве аргументов и вызывает подходящие функции для определения того, пересекаются ли эти фигуры. Для решения этой задачи вам придется добавить подходящие (виртуальные) функции к классам. Не пишите код, проверяющий пересечение; просто убедитесь, что вызываются правильные функции. Такой подход часто называют двойной диспетчеризацией или мульти-методом.

3. Реализовать класс «Человек», включающий в себя имя, фамилию, отчество, год рождения и методы, позволяющие изменять/получать значения этих полей.

Реализовать производные классы:

1) «Преподаватель университета» с полями: должность, ученая степень, специальность, список научных трудов (массив строк);

2) «Член комиссии» с полями: название комиссии, год назначения в комиссию, номер свидетельства, автобиография (массив строк);

3) «Преподаватели – члены комиссии» (производный от 2 и 3). Дополнительное поле – список работ выполненных в комиссии.

Классы должны содержать методы доступа и изменения всех полей.

4. Реализовать класс «Человек», включающий в себя имя, фамилию, отчество, год рождения и методы, позволяющие изменять/получать значения этих полей.

Реализовать производные классы:

1) «Предприниматель» - содержит номер лицензии, адрес регистрации, УНН, данные о налоговых платежах (массив пар вида <дата, сумма>).

2) «Турист» - содержит данные паспорта(строка), данные о пересечении границы в виде массива пар <дата, страна>.

3) «Челнок» (производный от 2 и 3) – добавляется массив строк – список адресов, по которым покупается товар

Классы должны содержать методы доступа и изменения всех полей.

5. Разработать класс Fakultet, порождающим класс Student. Класс Fakultet включает компоненты данные: наименование факультета, Класс Student, включает компоненты данные: Ф.И.О. студента, год рождения, результаты сдачи последней сессии.

6. Разработать иерархию классов для нахождения корней квадратного уравнения $ax^2+bx+c=0$. A – базовый для класса B, B – базовый для класса C. Компоненты классов A,B,C – коэффициенты a, b, c. Корни уравнения и методы решения находятся в классе C.

7. Создать базовый класс «транспортное средство» и производные классы «Автомобиль», «Велосипед», «Повозка». Подсчитать время и стоимость перевозки пассажиров и грузов каждым транспортным средством.

8. Создать базовый класс «грузоперевозчик» и производные классы «Самолёт», «Поезд», «Автомобиль». Определить время и стоимость перевозки для указанных городов и расстояний.

9. Создать аналогичный базовый класс «Пассажироперевозчик» и производные классы «Самолёт», «Поезд», «Автомобиль». Определить время и стоимость передвижения.

10. Создать базовый класс «Учащийся» и производные классы «Школьник» и «Студент». Создать массив объектов базового класса и заполнить этот массив объектами. Показать отдельно студентов и школьников.